Meta-Learning with Reptile

Fast Adaptation in Few-Shot Regression and Classification

Continual Learning Project (A/y 2025-26)

Shaikh Asif Hossain
Sorana-Ioana Resiga

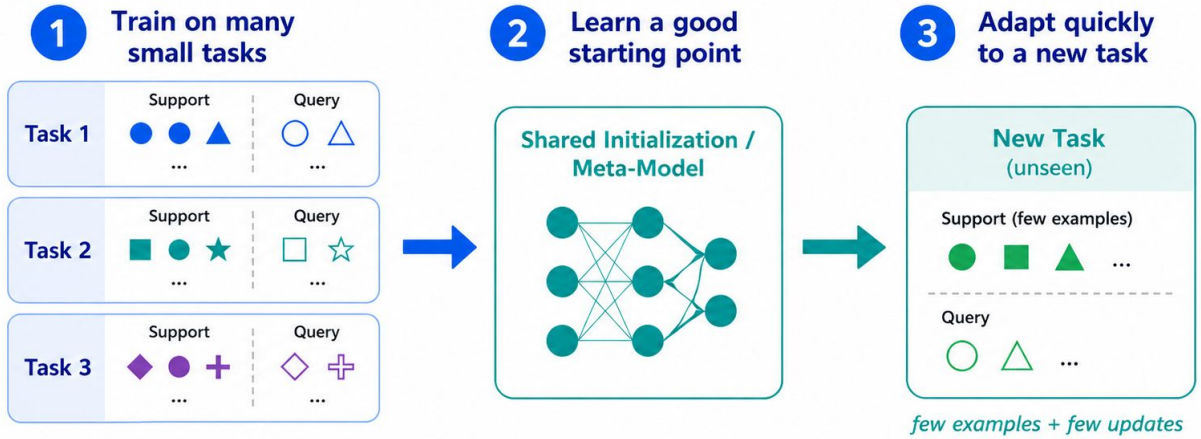
University of Pisa | Department of Computer Science

Motivation: Learning to Learn

The Meta-Learning Objective

Deep learning models usually require large amounts of labelled data. In contrast, humans learn from few examples.

Meta-learning aims to learn an **initialisation or procedure** that adapts quickly to new tasks from a related distribution.



Meta-learning trains a model so it can learn new tasks quickly from only a small amount of data.

Experiments: Sine Wave Regression & Omniglot Classification

Main Project Goals

- Understand theory and intuition behind Reptile
- Implement inner-loop and outer-loop update mechanisms
- Evaluate on sine wave regression and Omniglot few-shot classification
- Compare hyperparameter configurations and analyze inner-loop semantics

Theory: Meta & Few-Shot Learning

Meta-Learning

Learns from a distribution of tasks rather than a single task.

Goal: Quick adaptation to new tasks using few examples.

For each sampled task:

- Start from initialization (φ)
- Adapt to task-specific parameters ($\tilde{\varphi}$)
- Evaluate on unseen query samples

Few-Shot Learning

N-way K-shot Tasks

- **N** = number of classes per task
- **K** = support examples per class

Example: 5-way 5-shot

- 5 classes
- 5 labeled examples per class
- 25 support images total

Omniglot Experiments

Evaluating various configurations:

- **5-way 1-shot**
- **5-way 5-shot**
- **20-way 1-shot**
- **20-way-5-shot**

Reptile Algorithm: Workflow & Insights

The Reptile Algorithm

A first-order algorithm that learns neural network parameter initialisations. It moves original weights toward task-specific weights.

Computationally simpler than MAML as it avoids differentiating through the entire inner loop.

Algorithm 1: Reptile, serial version

Initialise ϕ, initial params of the model;
for $iteration = 1$ **to** n_{epochs} **do**
  Sample task τ with loss L_τ;
  $W \leftarrow \mathrm{SGD}(L_\tau, \phi, k)$, where k is the number of SGD steps;
  Do the update $\phi \leftarrow \phi + \epsilon(W - \phi)$;
end

*Notes by Vitaly Kurin https://yobibyte.github.io/

Serial Reptile Update

1. Start from initialization (φ)
2. Sample task & perform **k inner-loop** updates
3. Obtain adapted parameters ($\tilde{\varphi}$)
4. Update meta-parameters:

$$\varphi \leftarrow \varphi + \varepsilon(\tilde{\phi} - \varphi)$$

Learns an initialization that adapts quickly to new tasks.

Batched Reptile

1. Sample multiple tasks in one meta-batch.
2. Adapt one model copy per task using inner-loop updates.
3. Update the meta-model toward the average adapted direction for a more stable Reptile update.

Sine Wave Regression: Proof-of-Concept for Reptile

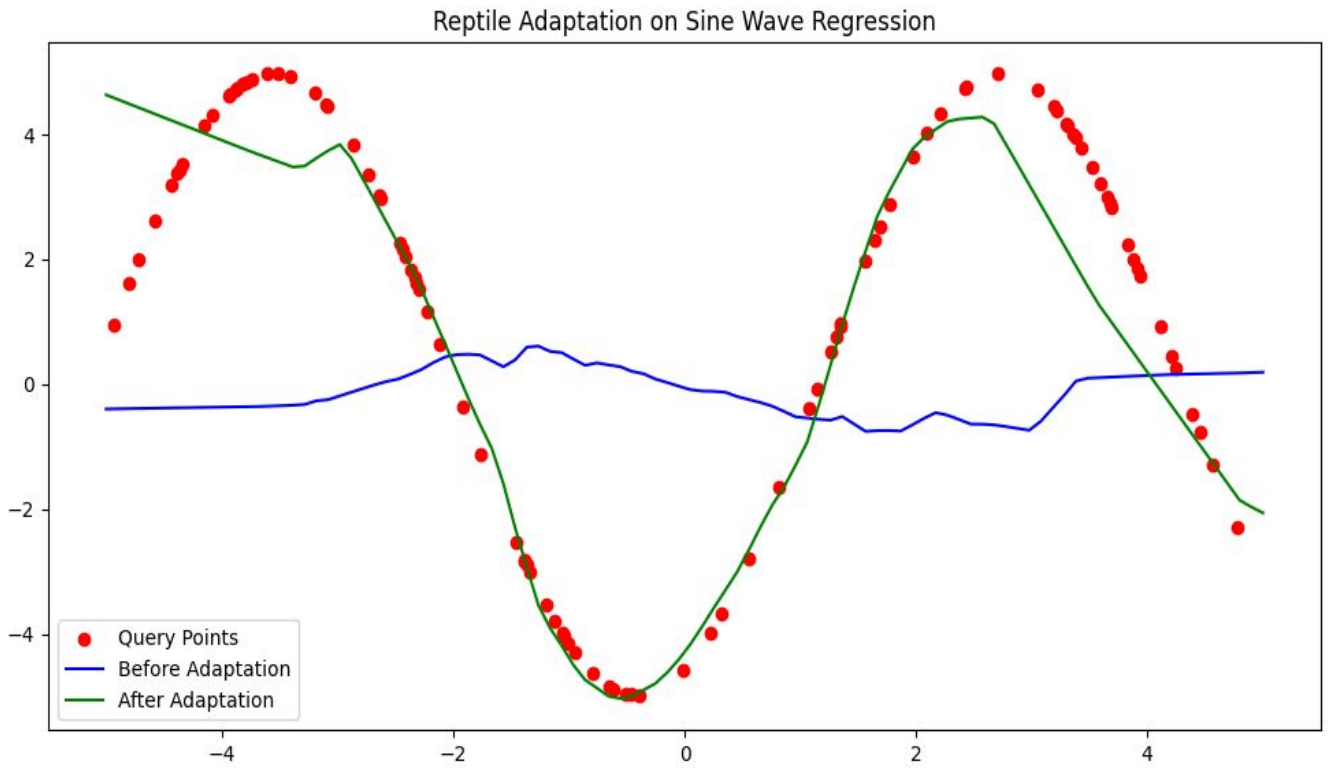
WHAT IT TESTS

- **Random Sine Tasks:** **Random Sine Tasks:** Each task is a different sine curve. The amplitude `A` is randomly chosen from `0.1` to `5.0`, and the phase `phi` is randomly chosen from `0` to `2π`.
- **Fast Adaptation:** Tests whether the model can quickly adapt to new sine waves instead of memorizing one fixed curve.
- **Support/Query Setup:** The model adapts on a small support set and is evaluated on query points, testing few-shot generalization.

HOW IT WORKS

- **Lightweight Model:** A small MLP with architecture `1 → 64 → 64 → 1` was used, allowing fast adaptation on each sine wave task.
- **Inner Optimization:** The copied model is adapted on the support points using vanilla SGD, with MSE as the regression loss.
- **Outer Optimization:** The original model is updated by moving its parameters toward the adapted model parameters using the first-order Reptile update.

Reptile Parameter Optimization Tracking Viewport



Core Takeaway: After a few localized support set gradients updates, the initialised parameters move efficiently from a flat, generic default state (Blue Line) to track highly volatile task-specific mappings (Green Line).

Omniglot Setup & Few-Shot Task Sampling

Dataset Infrastructure

- **Consolidated Pool:** Background and evaluation folders were merged into a single pool; each character class acts independently.
- **Class-Level Splitting:** The partitions are strictly divided at the character-class level, guaranteeing mutually exclusive domains and zero data leakage.

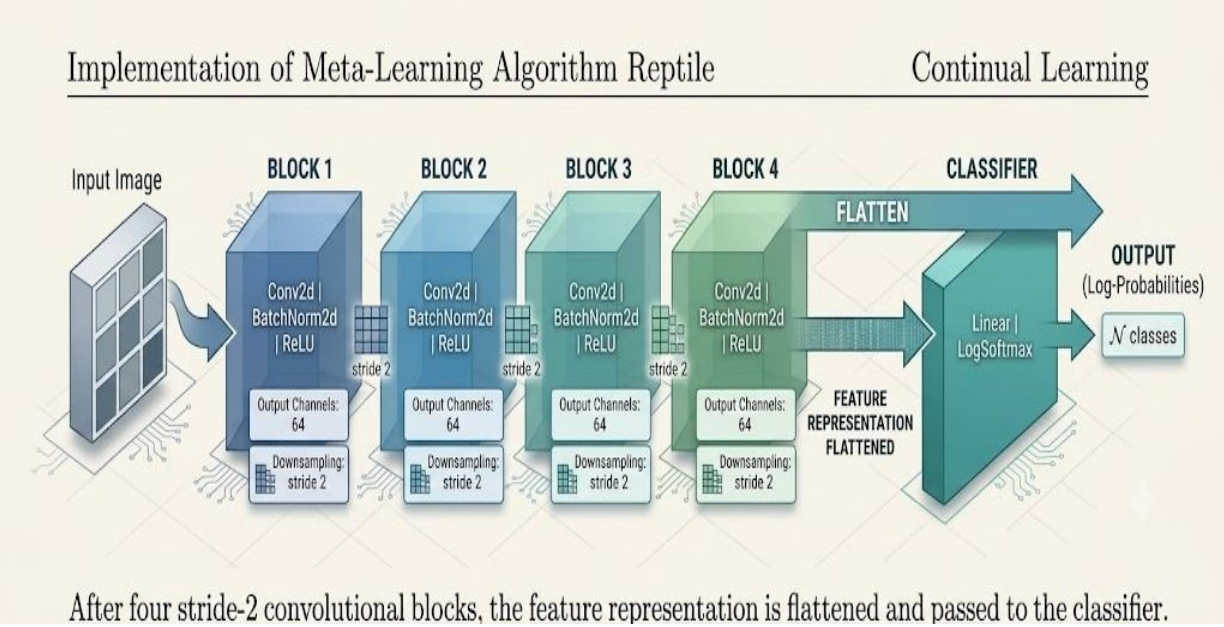
Episodic Task Generation

- **N-Way Mapping:** Randomly draws n_way classes per task, assigning temporary local indices (0, 1, ..., n-1).
- **K-Shot Split Strategy:** Out of the images per class, the first k_shot instances construct the adaptation support set; the rest form the query evaluation set.

Representation of the Omniglot character domain and subset separation

Example (5-Way 5-Shot): Dynamically isolates 5 target classes. Local inner loops train/adapt parameters using 25 total support images (5 per class), and generalization properties are evaluated across 25 disjoint query images.

CNN Architecture & BatchNorm Strategy



Backbone Architecture

- **Learned Downsampling:** Replaces traditional static Max Pooling operations with active stride=2 convolutions, enabling the neural network to autonomously learn its own optimized downsampling filters.
- **Progressive Compression:** The CNN compresses each 28×28 character image through four convolutional blocks into a compact $64 \times 2 \times 2$ feature map, which is flattened into a 256-dimensional feature vector.
- **Few-Shot Stability:** Strictly embeds BatchNorm2d immediately after each convolution layer to mitigate internal covariate shift during rapid, highly volatile inner-loop task updates.

⚠ **Buffer Corruption Prevention:** Restricting updates to parameters prevented contaminating non-learnable BatchNorm tracking statistics—such as running_mean and running_var—ensuring that localized task behaviors did not bleed across distinct training episodes or destabilize global initializations.

Inner & Outer Loop Strategy

Inner Loop: Adam for Task Adaptation

- For each sampled task, the meta-model is copied to initialize a local task_model.
- The cloned model is trained on the task's support set to perform rapid localized adaptation.
- Adam optimizer is utilized in the inner loop to accelerate parameter adjustments using few examples.
- Configured with $\beta_1 = 0$ and $\beta_2 = 0.999$; removing momentum isolates task-specific updates.

Outer Loop: Reptile Meta-Update

- Compares original meta-model parameters with adapted task-specific weights after inner loop completion.
- The global meta-model is shifted toward the adapted weights via a directional interpolation step.
- Executes an SGD-style update where the optimized parameter difference serves as the pseudo-gradient.
- The meta learning rate acts as the global step size, regulating initialization velocity.

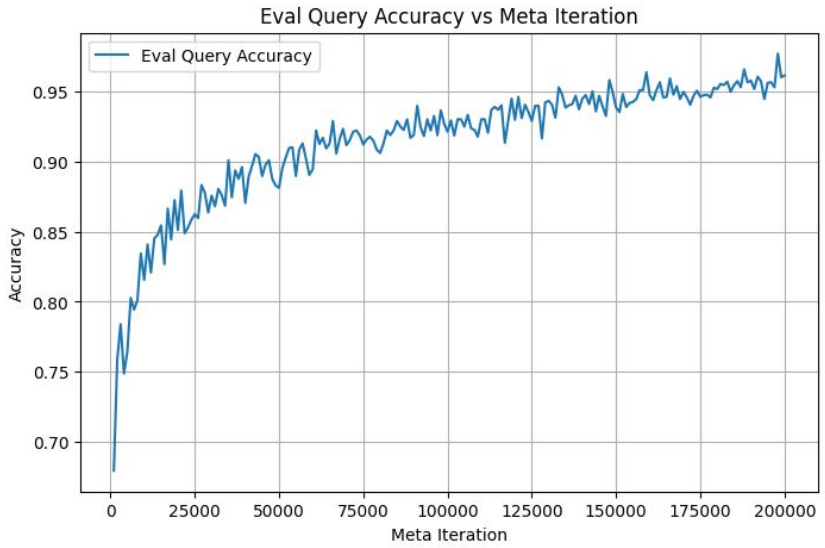
Main Idea: Adam is used to learn each specific task quickly in the inner loop, while the outer Reptile update extracts shared structural invariants to learn a robust global initialization for future tasks.

Final Configurations Summary & Results

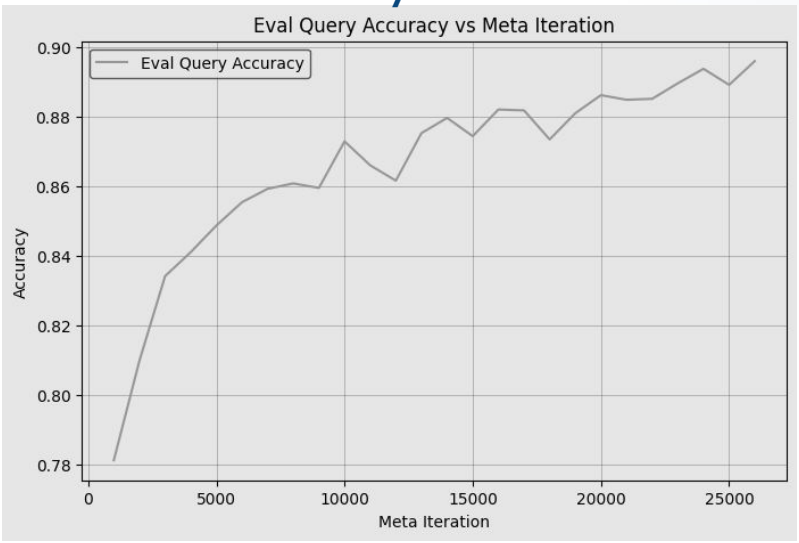
Setting	Meta LR	Inner LR	Iter.	Normalization	Val Acc	Test Acc	Query Loss
Sine Regression	0.1	0.01	70k	N/A	N/A	N/A	N/A
5-Way 1-Shot	0.33 -> 0.0	0.00044	200k	BatchNorm	**97.68%**	**93.55%**	**0.2325**
5-Way 5-Shot	0.2 → 0.0	0.001	100k	BatchNorm	**98.24%**	**97.23%**	**0.1212**
20-Way-1-Shot	1.0 → 0.0	0.4	100k	BatchNorm and Maxpool	**89.58%**	**85.27%**	**0.5662**
20-Way 5-Shot	0.2 → 0.0	0.0005	50k	BatchNorm	**92.29%**	**88.39%**	**0.4914**

Evaluation Accuracy Performance Plots

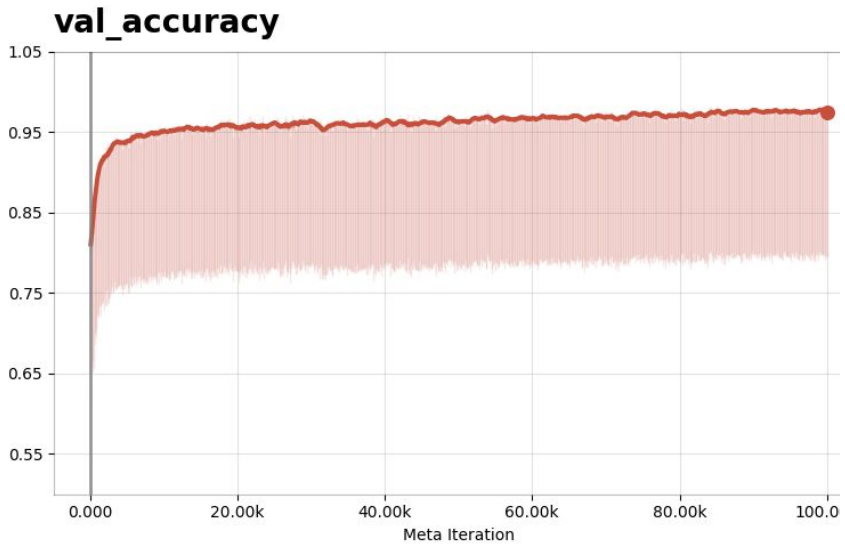
5-Way 1-Shot



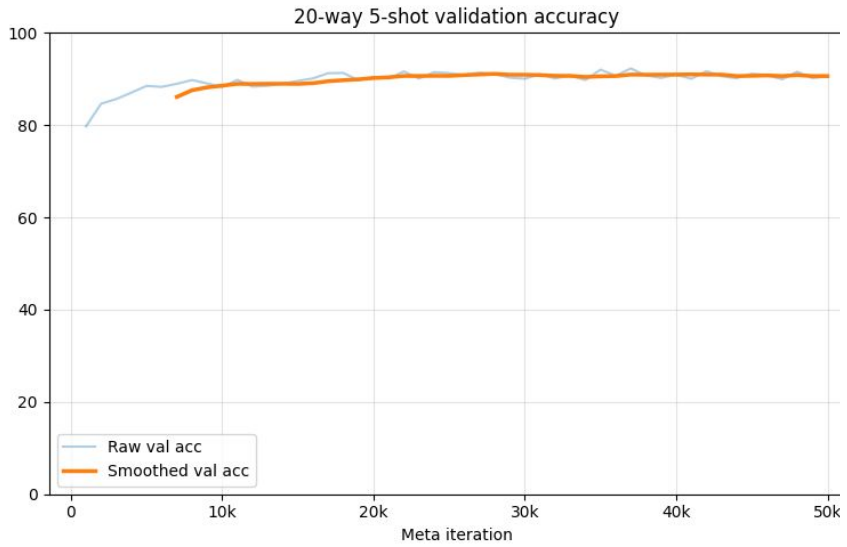
20-Way 1-Shot



5-Way 5-Shot



20-Way 5-Shot



Discussion : Implementation Challenges & Solutions

Challenge: Inner-Loop Step Misinterpretation

Initially, inner loop "steps" were treated as full epochs/passes over the support set.

- ✕ **Consequence:** This caused too many optimisation updates when support sets were split into mini-batches.
- ✕ **Impact:** The model overfitted strongly to the support sets, making validation and query loss curve behaviour highly unstable.

The Solution:

- ✓ Corrected the meaning of `inner_steps`.
- ✓ One inner step = one mini-batch optimiser update.
- ✓ This made validation/query behaviour more stable.

Challenge: Meta-LR Plateaus & Session Limits

Fixed meta-learning rates were either too slow or caused catastrophic training collapse.

- ✕ Early runs proved that standard 10K iterations plateaued before acquiring robust generic features.
- ✕ Kaggle session limits repeatedly cut off long-run execution paths.

The Solution:

- ✓ Implemented **Linear Learning Rate Annealing** (0.2 → 0.0 for 5-way; 1.0 → 0.0 for 20-way) alongside serialization checkpoints to allow 100K+ iterations.

⚠ **Analysis Insight:** Reptile is simple mathematically, but strong performance depends heavily on correct inner-loop semantics, stable meta-learning-rate scheduling, and long enough training.

Architectural Evolution & Limitations

Architectural Refinements

- Replaced max pooling with stride-2 convolutions.
- This allowed the CNN to learn downsampling directly from the data.
- The 28×28 input was compressed into a compact $64 \times 2 \times 2$ feature map.
- BatchNorm2d was used after convolution layers to stabilize rapid task-specific adaptation.
- Reptile meta-updates were applied only to learnable parameters, avoiding corruption of BatchNorm running statistics.

Remaining Challenges & Limitations

- **Hardest Omniglot setting:** The model must classify between 20 classes using only 1 support image per class, so the task has very limited adaptation information.
- **Performance gap remains:** The final test accuracy reached **85.27%**, which is far above the 5% random baseline, but still lower than the original Reptile paper's 20-way 1-shot result.
- **High computational cost:** Each 20-way task contains more classes and larger query evaluation, so validation/testing is much slower than the 5-way experiments.
- **Single-run evaluation:** The result was reported from one main run without multiple random seeds, so confidence intervals or statistical stability were not measured.

Future Direction: Improve the 20-way 1-shot setting by running longer training, testing multiple random seeds, performing wider hyperparameter tuning, and extending evaluation to harder datasets such as Mini-ImageNet.

Conclusion & Empirical Validation

Did We Expect These Results?

Yes and No: While theory guarantees that first-order meta-gradients work, the actual performance gaps across setups highlight interesting optimization dynamics:

- **Expected Success:** The sine wave setup smoothly developed an initialization optimized for quick adaptation. Similarly, the 5-Way classification boundaries produced strong, predictable trends, peaking at a remarkable test accuracy.

- **The Complexity Wall:** Scaling up to the 20-Way 5-Shot benchmark proved much harder. Reaching 88.39% clearly cleared the 5% random baseline, but left a notable performance gap compared to the paper, highlighting how sensitive meta-initialization space is to task density.

Empirical Validation Challenges

Achieving high final accuracy required identifying and fixing several core implementation hurdles:

- **Architectural Adjustments:** Replacing Max Pooling with stride-2 learned downsampling while treating BatchNorm buffers carefully was critical for stability.
- **Inner-Loop Steps:** Ensuring exactly one step meant one mini-batch update, rather than a full pass over the support set, was essential to stop the model from overfitting.
- **Long-Horizon Training:** Extending the meta-training cycle well beyond initial attempts alongside meta-LR annealing (0.2/1.0 → 0.0) unlocked successful convergence.

Final Verdict: The project confirms that while Reptile is mathematically simple to write, its empirical validation is highly sensitive to implementation details. Tuning the relationship between inner loops and outer loops is what turns moderate configurations into highly adaptable models.

References

[1] Alex Nichol, Joshua Achiam, and John Schulman. *On First-Order Meta-Learning Algorithms*. OpenAI, arXiv:1803.02999, 2018.

[2] Chelsea Finn, Pieter Abbeel, and Sergey Levine. *Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks*. International Conference on Machine Learning (ICML), 2017.

[3] Brenden M. Lake, Ruslan Salakhutdinov, Jason Gross, and Joshua B. Tenenbaum. *One Shot Learning of Simple Visual Concepts*. Proceedings of the Annual Meeting of the Cognitive Science Society, 2011.

[4] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. International Conference on Learning Representations (ICLR), 2015.

[5] Vitaly Kurin. *Reptile: A Scalable Meta-learning Algorithm*. Lecture note / summary, 2018.

Thank you